



-B-I-F-R-O-S-T

Federated analysis across TREs with different APIs

Team 1 · Nordic Biobank X NVIDIA 2026 hackathon · midterm POC



hunt · REST



gefion · DataSHIELD



brev · SQL

Different APIs and schemas across three TREs

The analysis must combine results accurately while keeping individual records inside each TRE.



TRE hunt

REST

```
POST /query
{"cols": ["rs001_gt"],
 "filters": {"alder": {">=": 60}},
 "agg": "value_counts"}
```

local columns

alder · kjon · bmi_baseline · rs001_gt



TRE gefion

DataSHIELD-style

```
POST /ds
{"fn": "ds.table",
 "args": {"x": "D$SNP_1"}}

(nfilter.tab = 3 applied
first)
```

local columns

age_years · sex_male · body_mass_index · SNP_1



TRE brev

SQL gateway

```
POST /sql
{"sql": "SELECT g_rs001,
count(*)

FROM cohort
GROUP BY g_rs001"}
```

local columns

AGE · SEX · BMI · g_rs001

Research questions: allele frequency of rs001 across all cohorts, and SBP ~ age + sex + BMI + LDL + 3 SNPs. Both use canonical variable names.

Design decisions

Shared adapters and site configuration support expansion beyond the initial three TREs.



Four adapter operations

Each TRE adapter implements count, describe, value_counts and gram through its local API. Analyses combine these operations.



Outbound TRE connections

A FLARE server coordinates clients inside each TRE. Clients connect over outbound mTLS gRPC, with no inbound ports required. We understand that inbound ports to TREs are impractical.



Site configuration

sites.yaml generates Docker Compose and FLARE configuration, kits and tests. One command adds a TRE. Region metadata supports future hierarchical aggregation.



Variable mapping

Researchers use names such as age, sex and snp_rs001. canonical.yaml maps these to local columns. A hash of the JSON spec identifies the analysis in the logs.



Disclosure checks and review

The TRE filters outputs and records an audit. The server checks merged results for k-anonymity, dominance and differencing. An overseer reviews flagged results.

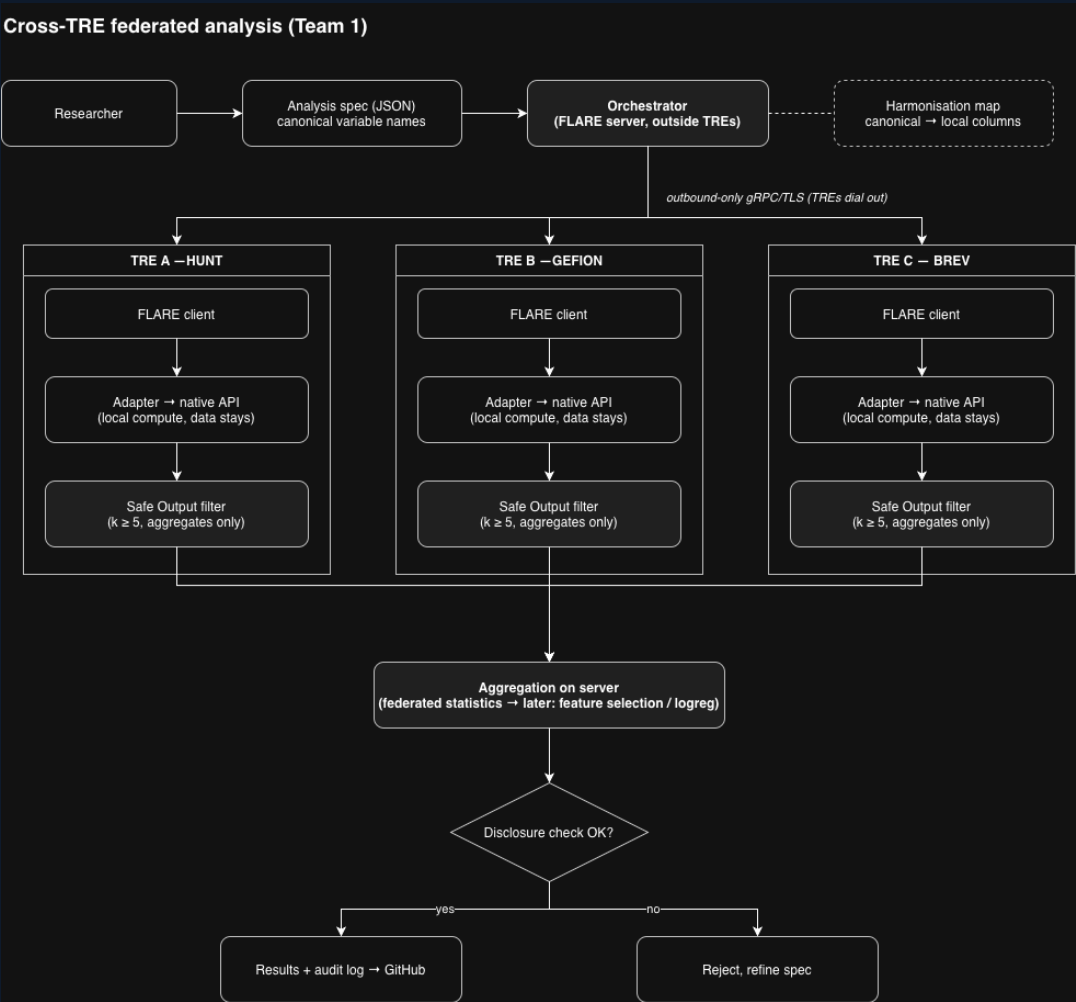


Aggregate results

TREs return counts, sums, Gram matrices or model parameters. The server combines available results using a minimum site count and a wait time.

System architecture

The implementation follows the team's initial architecture.



Request

AnalysisSpec: type, canonical variables, filters, min_cell_size, project_id.
Validated before any client runs.



Dispatch

The FLARE controller broadcasts the spec and collects results according to min_clients and wait_time.



Local execution

Executor loads the adapter for its own tre_id (FLARE identity = tre_id) and calls the native API.



Safe output

The TRE filters results before passing them to the FLARE client and records each decision in audit.jsonl.



Aggregation and review

The server combines results and checks disclosure risk. It writes released.json or sends the result for review.

TRE-Adapter

Each adapter translates the local API response into an AggregateResult.

```
class TREAdapter(ABC):
    def schema(self)                # canonical ->
dtype
    def count(self, filters) -> int
    def describe(self, col, filters) # n sum sum_sq min
max
    def value_counts(self, col, filters) # {level: n}
    def gram(self, cols, filters)       # [1,y,X]^T
[1,y,X]

    def run(self, spec) -> AggregateResult
        # shared, never overridden:
        #   canonical -> local   (canonical.yaml)
        #   primitives in the TRE's dialect
        #   Safe Output filter + audit line

registry.load("gefion")    # sites.yaml -> datashield
```



API implementations

rest.py, datashield.py and sql.py are 40–60 lines each. DataSHIELD nfilter.tab suppressions retain the label tre_native.



Regression uses gram()

Federated linear regression (M4) reuses gram(), which also supports means and variances. The existing adapters needed no changes.



Runtime support

The adapter, executor and job builder run in unit tests, the FLARE simulator, separate FLARE processes and Docker containers.

Allele frequency results

spec/examples/allele_freq.json, run through FLARE and checked against pooled data

SNP	hunt	gefion	brev	federated	ground truth
snp_rs001	0.0619	0.1050	0.0612	0.074617	0.074617
snp_rs002	0.1913	0.3385	0.2045	0.237933	0.237933
snp_rs003	0.1852	0.3259	0.1994	0.230083	0.230083
snp_rs004	0.1228	0.2193	0.1372	0.154550	0.154550
snp_rs005	0.1058	0.1880	0.1116	0.131500	0.131500
n	14 892	8 953	6 155	30 000	30 000

The synthetic cohorts have different allele frequencies and local column names. Summing counts across sites gives the pooled frequency to floating-point precision.

0.0

absolute error against pooled data for all 5 SNPs

11.9 s

from submission to merged result (federated round: 1.4 s)

2/3

sites reported after brev’s container stopped. Results matched the pooled data from those two sites.

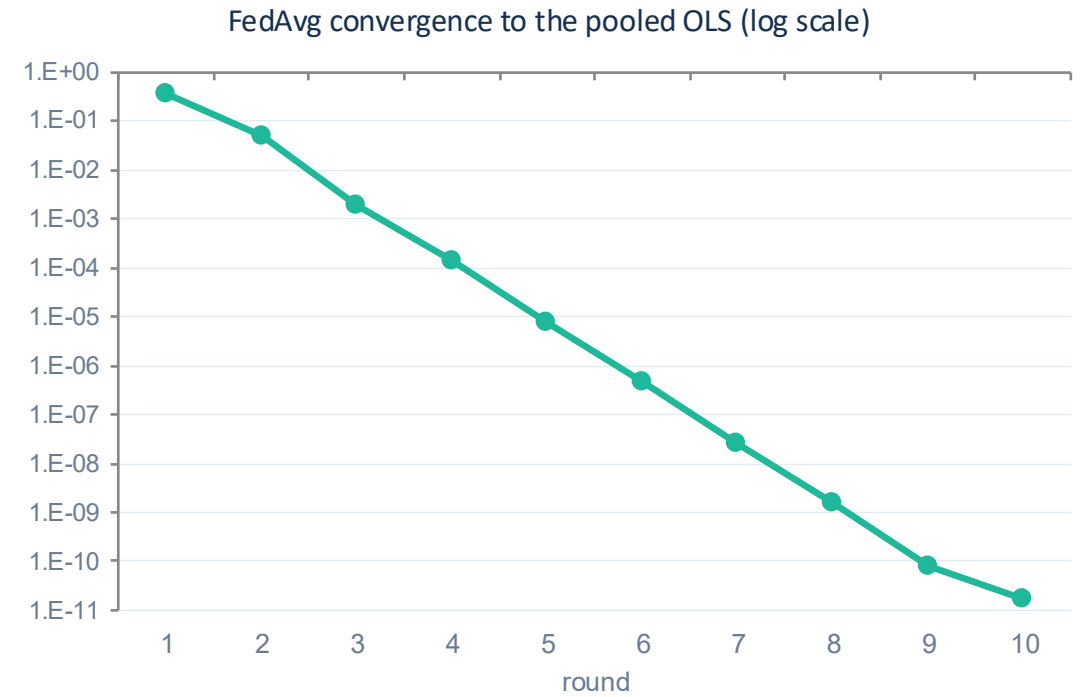
Linear regression results

$sbp \sim age + sex + bmi + ldl + rs001 + rs007 + rs013$

coefficient	exact (1 round)	FedAvg (10 rounds)	pooled OLS
intercept	89.2729	89.2729	89.2729
age	0.5055	0.5055	0.5055
sex	4.2034	4.2034	4.2034
bmi	0.8062	0.8062	0.8062
ldl	1.2189	1.2189	1.2189
snp_rs001	2.3934	2.3934	2.3934
snp_rs007	-1.2817	-1.2817	-1.2817
snp_rs013	0.9003	0.9003	0.9003
max error	1.7e-11	1.8e-11	—

Exact: sites return Gram matrices $[1, y, X]^T [1, y, X]$. The server sums them and solves for the coefficients.

FedAvg: each site keeps its Gram matrix locally and sends 8 coefficients per round. Local updates use a full-batch gradient step on sufficient statistics.



With 5 local steps per round, FedAvg's coefficient error was approximately $2.4e-11$ on the non-IID data. This case remains in the tests.

Output checks and release review

With min_cell_size = 100, all sites withheld the homozygous-minor cell and its derived allele counts.

audit/gefion/audit.jsonl records each spec at the site

```
{
  "tre_id": "gefion",
  "spec_hash": "bb552dd558d1d39c",
  "analysis_type": "allele_freq",
  "variables": ["snp_rs001"],
  "min_cell_size": 100,
  "n": 8953,
  "released": {"snp_rs001": ["genotype_counts"]},
  "rejected": [
    "snp_rs001.genotype_counts.2:count=89<100",
    "snp_rs001.allele_counts:derived_from_suppressed_cell"],
  "decision": "PARTIAL"
}
```

server: overseer queue

```
$ python -m server.overseer_queue list
1 pending: bb552dd558d1d39c site_suppression:hunt:2; gefion:2; brev:2
$ python -m server.overseer_queue approve bb552dd558d1d39c --by lead \
  --note "hom-minor cell withheld by design; 0/1 counts releasable"
bb552dd558d1d39c: APPROVED -> server/out/bb552dd558d1d39c/released.json
```

Five Safes controls



Safe projects

The TRE checks project_id against projects.yaml and logs refusals before computation.



Safe people

Each client and admin has an mTLS certificate signed by the project CA.



Safe settings

TRE containers and data mounts are read-only on internal networks. FLARE clients connect outbound. Tests confirmed blocked internet access from TREs.



Safe data

Adapters request aggregates. Each mock API independently rejects requests for individual records.

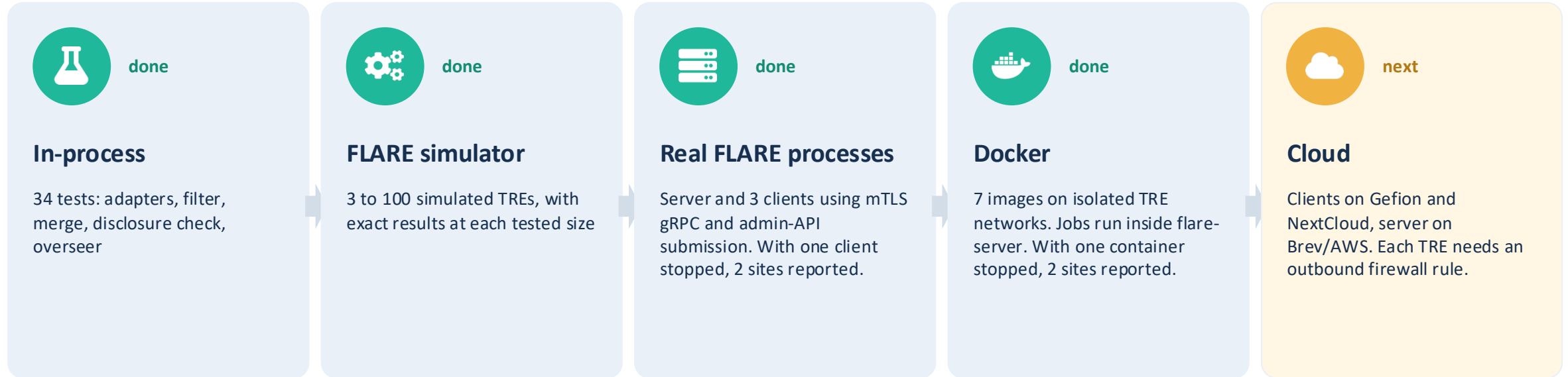


Safe outputs

Site filters enforce the allow-list and cell thresholds. Server checks cover k-anonymity, dominance, differencing and a minimum of 2 sites. Flagged results go to an overseer.

Testing and deployment status

Tests cover local execution, FLARE and Docker. Cloud deployment is next.



```
$ scripts/onboard_tre.sh gefion datashield https://opal.gefion.example/api nordic
sites.yaml: added gefion · compose + FLARE project regenerated · provisioned · flare/kits/gefion.tgz packed (1.4 s)
Firewall rule the TRE must allow (outbound ONLY, nothing inbound):
    allow TCP egress -> flare-server.brev.example:8002 (FLARE fed_learn, gRPC over mTLS)
```

Scaling test: 3 to 100 TREs

scripts/scale_sim.py tests allele frequency using REST, DataSHIELD and SQL adapters across four regions.



100/100

sites reported, with an error of 0.0

< 1 ms

to sum results from 100 sites

1.4 s

to add a TRE and generate its configuration and kit

Runtime increased with FLARE simulator scheduling across 8 client threads. Merging 100 results took under 1 ms. Associative merging and site region metadata support a future regional relay tier.

What cloud implementation is setup now?

AWS

Working NVFLARE server-client setup



HUNT Cloud, Gefion, Brev

Preliminary connection



Next steps

Next are cloud deployment, real schemas and a researcher interface.



Cloud deployment

Configure the Brev server address and provision the Gefion and NextCloud clients. The laptop federation remains the demo fallback, with a 30 s startup.



Researcher interface

Build a page for selecting variables and running analyses, with results, coverage and audit records. Add an approval view for the overseer using the existing scripts.



Real variable names and variants

Replace the invented local column names and rs001–rs020 with each site's schema and real rsIDs with a fixed effect allele (docs/variables.md).



Hierarchical aggregation

Add regional relay aggregators through FLARE's hierarchical topology, using the existing associative merge.

Team 1 — Ioannis Christoflogiannis · Gaurang Sharma · Marta Menta Czinkoczky · Udogwu Emiri · Pedro Gabriel Campana · Vitalii Babenko · Melissa Wong · Espen Hagen

github.com/collaborativebioinformatics/Federated_APIS